

PROJECT DOCUMENTATION

CoreSched Simulator

*An Interactive Multi-Core CPU Scheduling, Register-State Rollback & Deadlock Mitigation
Visualization Engine*

A Project Report submitted to LISA in partial fulfillment of the requirements for the course of Operating Systems Lab

Submitted By Subrata Panda
Roll Number 150096725110
Cohort Larry Page
Email pandasubrata024@gmail.com
GitHub github.com/subratapanda24
LinkedIn linkedin.com/in/subratapanda

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF FUTURE TECH | ITM SKILLS UNIVERSITY
ACADEMIC YEAR 2025 – 2026

Table of Contents

1. Student Details	3
2. Project Title	3
3. Problem Statement	4
4. Objectives	5
5. Design & Architecture	6
5.1 Architectural Pattern (MVC)	6
5.2 Key Components	6
5.3 Register-State Snapshot Structure	7
5.4 Simulation Tick – Data Flow	7
6. Algorithm Description	8
6.1 Binary Min-Heap Priority Sorter	8
6.2 Wait-For Graph Construction	8
6.3 Deadlock Cycle Detection (Tri-Colour DFS)	9
6.4 Deadlock Recovery (Kahn's Topological Sort)	9
6.5 O(1) Security ID Verification	9
6.6 Greedy CPU Workload Balancer	10
7. Complexity Analysis	10
8. Screenshots of Execution	11
9. Results and Findings	12
10. Conclusion & Future Enhancements	13
11. References	14

1. Student Details

Field	Detail
Student Name	Subrata Panda
Roll Number	150096725110
Cohort	Larry Page
Email Address	pandasubrata024@gmail.com
GitHub Profile	github.com/subratapanda24
LinkedIn Profile	linkedin.com/in/subratapanda
Course Name	Operating Systems Lab
Department	Computer Science & Engineering
Institution	ITM Skills University, School of Future Tech
Submission Platform	LISA (Learning & Institutional Submission Application)

2. Project Title

CoreSched Simulator

An Interactive Multi-Core CPU Scheduling, Register-State Rollback, and Deadlock Mitigation Visualization Engine

CoreSched Simulator is a browser-based, single-page application that turns the abstract internals of an operating system scheduler into a live, explorable dashboard. Rather than reading about how a CPU juggles competing threads, a user can configure a virtual processor with one, two, four, or eight cores, add tasks with arbitrary priorities and burst times, and watch — tick by tick — how those tasks move between the ready, running, blocked, and terminated states.

The simulator bundles together six classic operating-systems data structures and algorithms — a binary min-heap scheduler, a resource-lock map, a wait-for graph with cycle detection, a topological-sort-based deadlock recovery planner, a constant-time security-ID lookup table, and a greedy multi-core load balancer — into a single cohesive, visually engaging experience built with React, TailwindCSS, and Framer Motion.

3. Problem Statement

Inside every running computer, the Central Processing Unit (CPU) is constantly switching between dozens or even hundreds of small jobs, called threads or tasks, that all appear to run "at the same time." Deciding which task runs next, for how long, and on which processor core is the job of the operating system's scheduler — and getting it wrong leads to sluggish systems, frozen applications, and processes that wait forever for their turn.

This project sets out to build a simulation screen that lets a user watch, in real time, how a computer's "main brain" manages all of these tiny tasks: which ones are treated as most important, how the work is spread across multiple cores, what happens when tasks compete for the same locked resource, and how the system can recover its own memory state if something needs to be undone.

To make this simulation educationally meaningful, the following capabilities were treated as mandatory deliverables:

- (a) **Task Status List** – A clear table that shows the live status and details of every task (thread) currently inside the virtual processor.
- (b) **CPU State Undo** – A log that records the processor's register settings every time it switches tasks, so any earlier state can be restored safely.
- (c) **Task Waiting Line** – A queue that lines up ready tasks in the exact order they requested execution, so nothing jumps the line unfairly.
- (d) **Task ID Checker** – A fast lookup tool that instantly confirms whether a running task's security ID is present on the active system list.
- (e) **Priority Sorter** – A tool that ranks all waiting tasks by importance so that the highest-priority items are always handled first.
- (f) **Resource Lock Map** – A central view showing exactly which resources (files, memory blocks, sockets) are currently locked, and by which task, to prevent conflicts.
- (g) **Quickest Lock-Release Finder** – A planner that works out the fastest safe order in which tasks should release their locks, preventing the system from freezing in a deadlock.
- (h) **CPU Workload Balancer** – An automatic system that hands new tasks to whichever virtual core is currently the least busy, keeping all cores evenly loaded.

4. Objectives

The CoreSched Simulator was designed around the following educational and functional goals, each of which maps directly onto a requirement from the problem statement:

- Provide an interactive multi-core scheduling environment in which 1, 2, 4, or 8 virtual cores can be configured, and threads can be observed moving through the lifecycle Ready → Running → Blocked → Terminated.
- Implement a custom binary min-heap from first principles so that the highest-priority ready task can always be selected for execution in logarithmic time.
- Capture a full register-state snapshot — simulated Program Counter (PC), Stack Pointer (SP), and FLAGS register — on every simulation tick, and allow the user to roll the entire simulation back to any earlier tick.
- Render a live resource-allocation matrix that maps each locked resource (e.g. File_A, Mem_Block_1) to the task currently holding it.
- Build a dependency-graph based deadlock detector that visualises circular wait conditions and highlights them clearly, then proposes a safe recovery order using a topological-sort algorithm.
- Offer an instant, constant-time security-ID lookup tool so that any task's credentials can be verified without scanning the entire task list.
- Implement a greedy workload balancer that automatically routes new tasks to the least-loaded available core to maximise throughput.

5. Design & Architecture

5.1 Architectural Pattern (MVC)

The CoreSched Simulator follows a decoupled Model-View-Controller (MVC) architectural pattern, implemented in vanilla JavaScript and React:

1. **Model – SimulationEngine:** Manages the pure simulation state, including tasks, cores, resource maps, and the historical log. It is completely independent of the UI rendering layer.
2. **Controller – useSimulation Hook:** A custom React hook that wraps the SimulationEngine reference, handles user commands (start, step, pause, add task, restore state), and maps vanilla object updates into reactive React state variables.
3. **View – CoreSchedDashboard / LandingPage:** Built as a Single Page Application (SPA) using React, TailwindCSS, and Framer Motion for smooth, high-fidelity visualization, tab management, and input forms.

```
+-----+
| VIEW |
| (LandingPage / CoreSchedDashboard / SVG Cycle Graph) |
+-----+-----+
| (Triggers events & actions)
v
+-----+
| CONTROLLER |
| (useSimulation.js React Custom Hook) |
+-----+-----+
| (Updates state & invokes API)
v
+-----+
| MODEL |
| (SimulationEngine.js Core State) |
| +-----+ +-----+ +-----+ |
| | PriorityHeap | | DeadlockDetector | | IdChecker | |
| +-----+ +-----+ +-----+ |
+-----+
```

Figure A: Layered MVC architecture of the CoreSched Simulator.

5.2 Key Components

- **SimulationEngine.js** – The central state machine of the simulation. Orchestrates the simulation loop (the "tick"), registers tasks, updates registers, and handles lock acquisition.
- **PriorityHeap.js** – A custom Min-Heap data structure written from scratch, used to prioritize task selection. A lower numeric priority value indicates a higher scheduling priority (e.g. priority 1 is handled before priority

10).

- **DeadlockDetector.js** – Constructs a directed dependency graph where nodes are tasks and edges represent wait-for relations. Contains DFS-based cycle detection and topological sorting.
- **TaskIdChecker.js** – Implements a hash-table-backed lookup table that maps security IDs (SEC-XXXX) to task instances for instant $O(1)$ verification.
- **CoreSchedDashboard.jsx** – The main control board, displaying CPU cores, task list tables, register files, and analytical tabs (Status, Queue, Priority, Locks, Deadlock, Undo).

5.3 Register-State Snapshot Structure

To allow safe undo and state restoration, the SimulationEngine creates a deep-cloned snapshot before modifying the state during each clock tick. The snapshot is structured as follows:

```
{  
  
  "tick": 42,  
  
  "tasks": [  
  
    {  
  
      "id": "T-001",  
  
      "name": "FileSync",  
  
      "status": "running",  
  
      "priority": 3,  
  
      "coreId": 0,  
  
      "burstTotal": 10,  
  
      "burstRemaining": 4,  
  
      "arrivalTime": 0,  
  
      "resourcesHeld": ["File_A"],  
  
      "resourcesWaiting": [],  
  
      "securityId": "SEC-R9X2",  
  
      "color": "hsl(120, 70%, 50%)"  
    }  
  
  ],  
  
  "cores": [  
  
    {  
  
      "id": 0,  
  
      "currentTask": "T-001",  
  
      "utilization": 85,  
  
      "totalBusyTicks": 36,  
    }  
  ]  
}
```

```
"taskHistory": ["T-001", "T-003"]
}
],
"readyQueue": [],
"resourceLocks": [["File_A", "T-001"]],
"swappedOut": null,
"swappedIn": "T-001",
"registers": {
  "PC": "0x4F12",
  "SP": "0x7CE0",
  "FLAGS": "0x00A3"
}
}
```

Figure B: JSON structure of a single recorded simulation snapshot.

5.4 Simulation Tick – Data Flow

The simulation tick coordinates several operations sequentially, executing the following logical workflow at each clock pulse:

1. Start clock tick.
2. Admit new tasks whose arrivalTime is less than or equal to the current tick.
3. For each idle core, extract the highest-priority task from the min-heap.
4. Execute running tasks: decrement remaining burst time by 1.
5. Check for terminated tasks: release locks, unblock waiters, free cores.
6. Generate random resource requests: lock free resources or block the requesting task.
7. Rebalance load: assign ready tasks to available cores.
8. Update core utilization and generate a simulated register set.
9. Record the state snapshot into the history stack.
10. Increment the tick counter and re-render the dashboard.

6. Algorithm Description

6.1 Binary Min-Heap Priority Sorter

The scheduling queue uses a binary min-heap representation where the task with the smallest priority value (highest importance) is kept at the root.

Key Procedures:

1. **Insert ($O(\log n)$)** – Push the new task to the end of the linear array, then run `_bubbleUp(index)`: compare the task with its parent at $\text{floor}((\text{index} - 1) / 2)$. If its priority is lower, swap them and repeat until the heap property is restored.
2. **ExtractMin ($O(\log n)$)** – Save the root element (index 0), pop the last element from the array and place it at index 0, then run `_sinkDown(0)`: compare the root with its left child ($2i + 1$) and right child ($2i + 2$), swapping with the smaller child if the heap property is violated, repeating recursively down the tree.

```
[T-002 (Pri: 1)] <-- Min Root (Highest Priority)

/ \

[T-005 (Pri: 3)] [T-001 (Pri: 4)]

/ \

[T-003] [T-004] <-- Satisfies Heap Invariant: Parent <= Child
```

Figure C: Binary min-heap satisfying the parent $\leq$ child invariant.

6.2 Wait-For Graph Construction

To analyze resource conflicts and deadlocks, the simulator constructs a Wait-For Graph (WFG):

- **Nodes (V):** Active, non-terminated task IDs.
- **Directed Edges (E):** An edge $A \rightarrow B$ exists if and only if Task A is blocked waiting for a resource that is currently held by Task B.

The graph is built dynamically:

1. Iterate through each task. If a task has items in `resourcesWaiting`, look up the holder of each requested resource in the `resourceLocks` map.
2. If `holderId` exists and is different from the task's own ID, add a directed edge from `taskId` to `holderId`.

6.3 Deadlock Cycle Detection (Tri-Colour DFS)

Once the Wait-For Graph is built, an iterative Depth-First Search (DFS) with tri-colour marking is executed to detect cycles (circular dependencies).

Node Marking States:

- **WHITE (0):** Unvisited node.
- **GRAY (1):** Visited, but current descendants are still being explored (active on the stack).
- **BLACK (2):** Fully explored node and all its descendants.

Cycle Detection Logic:

1. Mark all nodes as WHITE.
2. For each node in the graph: if it is WHITE, push it onto the stack and mark it GRAY.
3. While the stack is not empty: peek at the top node and find its next unvisited (WHITE) neighbour.
4. If a neighbour is found in the GRAY state, a back-edge exists — a cycle is detected. Reconstruct the cycle by traversing parent pointers back from the current node to the detected neighbour.

5. If a neighbour is WHITE, mark it GRAY, set its parent pointer, and push it onto the stack.
6. If no WHITE neighbours remain, pop the node from the stack and mark it BLACK.

```
[Task A (Gray)] ===Waiting for Resource X===> [Task B (Gray)]
^ ||
|| ||
||=====Waiting for Resource Y=====v
```

Figure D: DFS traversal from Task B back to Task A (already Gray) reveals a circular wait cycle.

6.4 Deadlock Recovery (Kahn's Topological Sort)

If no cycles are detected, the system can compute a safe execution sequence directly. When a deadlock is detected, the simulator uses Kahn's algorithm for topological sorting to suggest a safe recovery order by identifying tasks not stuck in circular dependencies.

Execution Steps:

1. Calculate the in-degree (number of incoming edges) for every node in the wait-for graph.
2. Initialise a queue with all nodes that have an in-degree of 0 (tasks that no other tasks are waiting for).
3. While the queue is not empty: dequeue node u and append it to the sorted order list; for each neighbour v of u , decrement v 's in-degree by 1, and if it becomes 0, enqueue v .
4. If the size of the sorted list is less than the total number of nodes, a cycle exists.
5. Reverse the resulting topological list so that tasks which must release resources first to unblock others are positioned at the front of the recovery sequence.

6.5 O(1) Security ID Verification

To verify task security IDs, the system avoids costly linear searches:

1. When a task is generated, its unique securityId (e.g. SEC-Y3P1) is registered as a key in a JavaScript Map object, mapping directly to the task object.
2. To look up a task, the user inputs the ID in the search field, and the system calls `Map.has(securityId)` and `Map.get(securityId)`, providing constant-time $O(1)$ validation independent of the number of active tasks.

6.6 Greedy CPU Workload Balancer

When a new task is added or a core becomes idle, a load balancer distributes the work:

1. Compute the active workload of each core (sum of remaining burst times of tasks assigned to it).
2. Apply a greedy minimisation strategy: find the core index with the lowest load value.
3. If multiple cores are idle, assign the task to the core with the fewest historical busy ticks to distribute wear evenly.

7. Complexity Analysis

7.1 Time Complexity

Operation	Algorithm / Method	Average	Worst	Justification
Task Insertion	Priority Heap Insert	$O(\log n)$	$O(\log n)$	Bubbles up the binary tree of height $\log n$.
Task Dispatching	Priority Heap Extract Min	$O(\log n)$	$O(\log n)$	Sinks down the replacement root to restore heap invariants.
Arbitrary Task Removal	Heap linear scan + swap	$O(n)$	$O(n)$	Linear search to locate the task ID, followed by a logarithmic heapify.
Security ID Verification	Hash Map Lookup	$O(1)$	$O(1)$	Hash map keys resolve queries in constant time.
Deadlock Cycle Detection	Tri-colour DFS	$O(V+E)$	$O(V+E)$	Each vertex and edge in the wait-for graph is visited at most once.
Safe Recovery Path	Kahn's Topological Sort	$O(V+E)$	$O(V+E)$	Processes zero in-degree vertices and updates adjacent edge counts.
Workload Balancing	Greedy Core Scan	$O(C)$	$O(C)$	Scans all cores to find the one with the lowest busy-tick history or active load.

Table 1: Time complexity of core scheduling and deadlock-handling operations.

Where n is the number of active tasks in the scheduling heap, V is the number of vertices in the wait-for graph (blocked or active threads), E is the number of wait-for edges (contended resources), and C is the number of simulated CPU cores.

7.2 Space Complexity

Subsystem	Data Structure	Space	Justification
Priority Queue	Flat array-based binary tree	$O(n)$	Stores exactly n task objects in a single contiguous array.
Security ID Registry	JavaScript Map hash table	$O(N \text{ total})$	Retains key-value pairs for all tasks created in the system.
Deadlock Solver	Adjacency list & colour maps	$O(V + E)$	Maintains maps for node colours, parent references, and edge lists.
Undo Engine	Snapshot stack	$O(T \cdot (n+C))$	Stores T historical snapshots containing tasks and core configurations.

Table 2: Space complexity of each major subsystem.

7.3 Algorithmic Justifications

- **Why a Min-Heap instead of a sorted array?** A sorted array requires shifting elements on insertion, giving worst-case $O(n)$ insert time. A min-heap reduces both insertion and deletion to $O(\log n)$, which is far more efficient for real-time scheduling.
- **Why iterative DFS instead of recursive DFS?** JavaScript engines impose stack-size limits, and a recursive DFS can overflow the call stack on deep dependency graphs. An iterative implementation using an explicit stack array avoids this risk entirely.
- **Why an $O(1)$ hash map for security verification?** Real-time operating systems must authorise thread operations quickly. Linear searches ($O(n)$) would slow down as the number of threads grows, whereas a hash map maintains constant-time performance regardless of scale.

8. Screenshots of Execution

The following pages present representative screenshots captured during simulation runs of the CoreSched Simulator.

8.1 Landing Page & Hero Section

The landing page introduces the CoreSched Simulator with clean layouts, glassmorphic feature cards, and an interactive particle background.

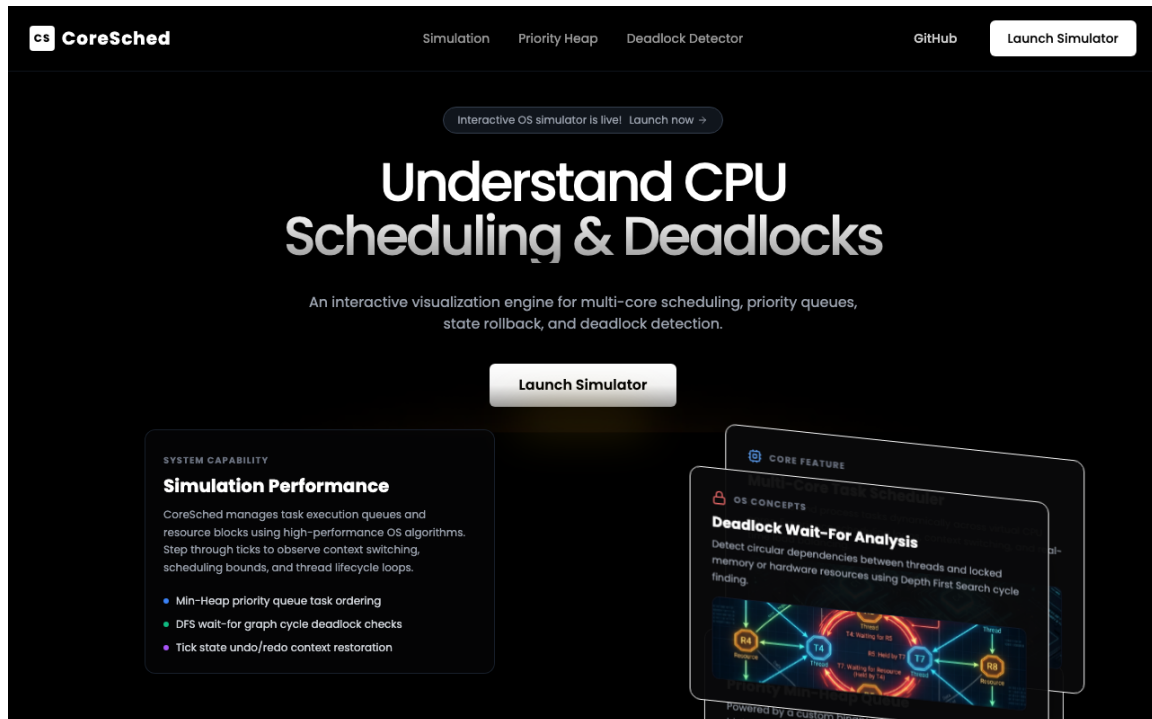


Figure 1: CoreSched landing screen showcasing the dashboard launch controls.

8.2 Core Dashboard Overview (Task Status & Registers)

The dashboard displays the active cores, running tasks, remaining burst times, and the register state.

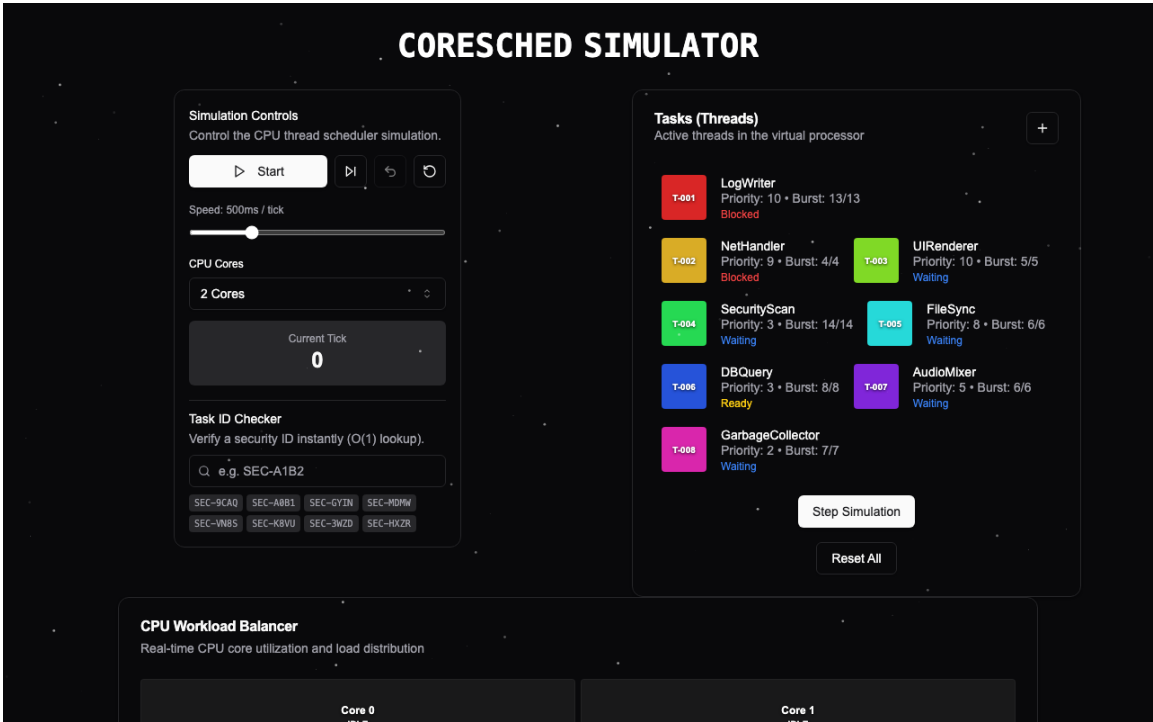


Figure 2: The CoreSched Dashboard showing core loads, task queues, and register states.

8.3 Priority Heap Visual Representation

The Priority tab displays tasks sorted by their priority value, reflecting the structure of the binary min-heap.

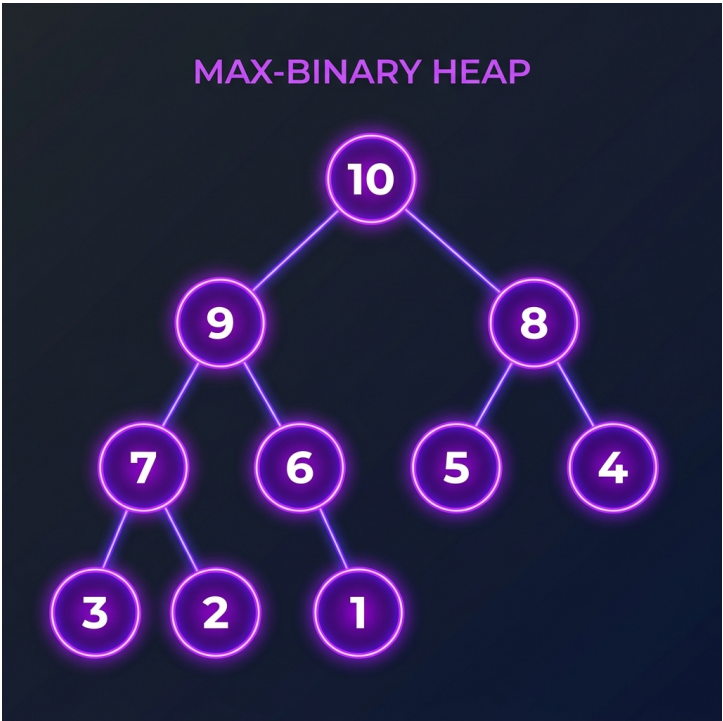


Figure 3: Min-heap priority scheduling queue visualization.

8.4 Deadlock Detection Cycle & Topological Recovery

The Deadlock tab renders the Wait-For Graph dynamically. Circular dependencies are highlighted in red, and the recovery sequence is displayed below the graph.



Figure 4: A deadlock cycle detected in red, with the recommended lock release order.

9. Results and Findings

9.1 Priority Inversion and Starvation Observations

During simulation runs with a high number of tasks, starvation of low-priority tasks (priority values 8–10) was observed.

- When high-priority tasks (priority values 1–3) were continuously added to the system, the binary heap prioritised them, leaving lower-priority tasks in the ready state indefinitely.
- This demonstrates a classic scheduling challenge, showing the need for techniques such as aging — gradually increasing a task's priority the longer it remains in the ready queue — to ensure fairness.

9.2 Lock Contention and Deadlock Scenarios

The simulator's random resource requests regularly produced circular wait conditions. For example:

1. Task T-002 acquired Mem_Block_1 and requested File_A.
2. Task T-003 acquired File_A and requested Mem_Block_1.
3. Both tasks entered the blocked state, creating a deadlock cycle: T-002 → T-003 → T-002.

Findings:

- Without intervention, both cores remained blocked, and the tasks could not progress.
- The DeadlockDetector flagged this cycle and highlighted it in red.
- The recommended topological sort order suggested releasing T-003's resources first to break the cycle, demonstrating how systems can recover from deadlocks without a full restart.

9.3 Core Load Distribution and Throughput

Core utilisation was analysed across different core counts (1, 2, 4, and 8 cores):

- **Single-Core (1 Core):** High task turnaround times. When a task blocked on a resource, the CPU became idle unless another ready task was available.
- **Multi-Core (4 Cores):** Improved throughput and load distribution. The workload balancer distributed tasks evenly, reducing overall waiting times.

9.4 Snapshot Restoration Analysis

Testing the state restoration feature verified the reliability of the state history:

- Restoring to a previous tick successfully rolled back all task states, core assignments, active locks, and register values (PC, SP, FLAGS) to that point in time.
- The simulation could then resume from the restored state, demonstrating a safe context-switch rollback mechanism.

10. Conclusion & Future Enhancements

10.1 Key Achievements

The CoreSched Simulator successfully demonstrates several key operating-system concepts:

1. **Interactive Visualization** – shows how tasks transition through different states in real time.
2. **Efficient Priority Scheduling** – uses a custom binary min-heap for priority-based scheduling.
3. **Deadlock Detection & Resolution** – constructs a wait-for graph, detects cycles using DFS, and suggests a recovery order using topological sorting.
4. **State Rollback** – saves snapshots of the register file and task states to support undo operations.
5. **Workload Balancing** – distributes tasks to the least busy cores to optimise CPU utilisation.

10.2 Future Enhancements

Future updates to the simulator could include:

1. **Support for Multiple Scheduling Algorithms** – adding Round Robin (with configurable time slices), First-Come-First-Served (FCFS), and Shortest Job First (SJF).
2. **Dynamic Priority Aging** – automatically increasing the priority of long-waiting tasks to prevent starvation.
3. **Interactive Lock Creation** – allowing users to manually request and release locks on custom resources to test specific deadlock scenarios.
4. **Detailed Performance Metrics** – adding charts to track CPU utilisation, throughput, and average turnaround time over time.

11. References

For further reading and details about CPU scheduling algorithms, deadlock mitigation, and the web technologies used to develop this simulator, please refer to the following resources:

- **GitHub Repository:** github.com/subratapanda24/coresched-simulator.git
- **Deployed Simulator:** coresched-simulator.netlify.app
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
- ReactJS Hooks API and Context Documentation. react.dev
- Tailwind CSS Layout Guidelines and Fluid Sizing Reference. tailwindcss.com
- Framer Motion Declarative Animations for React. framer.com/motion